

LAB SESSION 08

Binary Search Tree (BST)

A Binary Search Tree is a binary tree where:

1. Left subtree contains values smaller than the node
2. Right subtree contains values greater than the node
3. Both left and right subtrees are also BSTs

Algorithm: Insertion in BST

Start from root

If value $<$ root $\rightarrow$ go left

If value $>$ root $\rightarrow$ go right

Repeat until NULL

Insert node there

Algorithm

BST_Insert(root, key):

1. If root == NULL
 create new node
 return node
2. If key $<$ root.data
 root.left = BST_Insert(root.left, key)
3. Else if key $>$ root.data
 root.right = BST_Insert(root.right, key)
4. Return root

Algorithm: Searching in BST

Compare key with root

If equal $\rightarrow$ found

If smaller → search left

If larger → search right

Algorithm

BST_Search(root, key):

1. If root == NULL
 return NOT FOUND
2. If key == root.data
 return FOUND
3. If key < root.data
 return BST_Search(root.left, key)
4. Else
 return BST_Search(root.right, key)

Algorithm: Deletion in BST

Deletion has 3 cases:

Case 1: Node is Leaf

Just delete it.

Case 2: Node has One Child

Replace node with its child.

Case 3: Node has Two Children

Replace node with:

- Inorder Successor (minimum in right subtree)
 OR
- Inorder Predecessor (maximum in left subtree)

Algorithm

BST_Delete(root, key):

1. If root == NULL

return root

2. If key < root.data
root.left = BST_Delete(root.left, key)
3. Else if key > root.data
root.right = BST_Delete(root.right, key)
4. Else // Node found

Case 1: No child
return NULL

Case 2: One child
if root.left == NULL
return root.right
else if root.right == NULL
return root.left

Case 3: Two children
temp = MinValue(root.right)
root.data = temp.data
root.right = BST_Delete(root.right, temp.data)

5. Return root

Tasks:

Write a program to:

1. Insert elements into a Binary Search Tree
2. Search for a given value
3. Delete a node
4. Print Inorder Traversal (sorted order)

Sample Input

Insert: 50 30 70 20 40 60 80

Search: 60

Delete: 30